UNIT 3

Q: Differentiate between HAVING and where clause in SQL

1. WHERE Clause:

WHERE Clause is used to filter the data from the table or used while joining more than one table.Only those records will be extracted who are satisfying the specified condition in WHERE clause. It can be used with SELECT, UPDATE, DELETE statements.

HAVING Clause is used to filter the data from the groups based on the given condition in the HAVING Clause. Those groups who will satisfy the given condition will appear in the final result. HAVING Clause can only be used

with SELECT statement.

SR.NO.	WHERE Clause	HAVING Clause
1.	WHERE Clause is used to filter the records from the table based on the specified condition.	HAVING Clause is used to filter record from the groups based on the specified condition.
2.	WHERE Clause can be used without GROUP BY Clause	HAVING Clause cannot be used without GROUP BY Clause
3.	WHERE Clause implements in row operations	HAVING Clause implements in column operation
4.	WHERE Clause cannot contain aggregate function	HAVING Clause can contain aggregate function
5.	WHERE Clause can be used with SELECT, UPDATE, DELETE statement.	HAVING Clause can only be used with SELECT statement.
6.	WHERE Clause is used before GROUP BY Clause	HAVING Clause is used after GROUP BY Clause
7.	WHERE Clause is used with single row function like UPPER, LOWER etc.	HAVING Clause is used with multiple row function like SUM, COUNT etc.

Q: Explain the impedance mismatch problem. Which of the database programming approaches minimize this problem

Impedance mismatch is the term used to refer to the problems that occurs due to differences between the database model and the programming language model. The practical relational model has 3 components these are:

1. Attributes and their data types
2. Tuples
3. Tables

Problems: Following problems may occur due to the impedance mismatch:

1. The first problem that may occur is that is data type mismatch means the programming language attribute data type may differ from the attribute data type in the data model.
2. The second problem that may occur is because the results of most queries are sets or multisets of tuples and each tuple is formed of a sequence of attribute values. In the program, it is necessary to access the individual data values within individual tuples for printing or processing.

Q: Explain about union compatibility in SQL

Union compatible means
that the relations yield attributes with identical names and. **compatible**
data types. That **is, the**
relation A(c1,c2,c3) and the relation B(c1,c2,c3) have. **union compatibility**
if the columns have the same names, are in the same order, and the. columns have
“**compatible**
” data types.

id	first_name	last_name	salary
1	Ethan	Hunt	5000
2	Tony	Montana	6500
3	Sarah	Connor	8000
4	Rick	Deckard	7200
5	Martin	Blank	5600

Table: **employees**

id	first_name	last_name	city
1	Maria	Anders	Berlin
2	Fran	Wilson	Madrid
3	Dominique	Perrier	Paris
4	Martin	Blank	Turin
5	Thomas	Hardy	Portland

Table: **customers**

Let's perform a union operation to combine the results of two queries.

The following statement returns the first and last names of all the customers and employees:

Example — Try this code »

```
SELECT first_name, last_name FROM employees
UNION
SELECT first_name, last_name FROM customers;
```

After executing the above statement, the result set will look something like this:

first_name	last_name
Ethan	Hunt
Tony	Montana
Sarah	Connor
Rick	Deckard
Martin	Blank
Maria	Anders
Fran	Wilson
Dominique	Perrier
Thomas	Hardy

Q: Explain about any four aggregate functions in SQL

Now let us understand each Aggregate function with a example:

```
Id     Name     Salary
----------------------
1       A        80
2       B        40
3       C        60
4       D        70
5       E        60
6       F        Null
```

Count():

Count(*): Returns total number of records .i.e 6.
Count(salary): Return number of Non Null values over the column salary. i.e 5.
Count(Distinct Salary): Return number of distinct Non Null values over the column salary .i.e 4

Sum():

sum(salary): Sum all Non Null values of Column salary i.e., 310
sum(Distinct salary): Sum of all distinct Non-Null values i.e., 250.

Avg():

Avg(salary) = Sum(salary) / count(salary) = 310/5
Avg(Distinct salary) = sum(Distinct salary) / Count(Distinct Salary) = 250/4

Min():

Min(salary): Minimum value in the salary column except NULL i.e., 40.
Max(salary): Maximum value in the salary i.e., 80.

Q: Explain any three different clauses of SELECT in SQL with an example.

WHERE Clause

The WHERE clause specifies criteria that restrict the contents of the results table. You can test for simple relationships or, using subselects, for relationships between a column and a set of columns.

Using a simple WHERE clause, the contents of the results table can be restricted, as follows:

Comparisons:

```
SELECT ename FROM employee_dim
      WHERE manager = 'Al Obidinski';
```

GROUP BY Clause

The GROUP BY clause groups the selected rows based on identical values in a column or expression. This clause is typically used with aggregate functions to generate a single result row for each set of unique values in a set of columns or expressions.

A simple GROUP BY clause consists of a list of one or more columns or expressions that define the sets of rows that aggregations (like SUM, COUNT, MIN, MAX, and AVG) are to be performed on. A change in the value of any of the GROUP BY columns or expressions triggers a new set of rows to be aggregated.

If the GROUP BY clause contains CUBE or ROLLUP options, it creates superaggregate (subtotal) groupings in addition to the ordinary grouping.

GROUP BY has the following format:

```
GROUP BY [ALL | DISTINCT] grouping element {,grouping element}
```

HAVING Clause

The HAVING clause filters the results of the GROUP BY clause by using an aggregate function. The HAVING clause uses the same restriction operators as the WHERE clause.

For example, to return parts that have sold more than ten times on a particular day during the past week:

```
SELECT odate, partno, count(*) FROM orders
GROUP BY odate, partno
WHERE odate >= (CURRENT_DATE - INTERVAL '7' day)
HAVING count(*) > 10;
```

Q: Explain the purpose and syntax of the following commands in SQL by considering an relational schema that has at least one numerical attribute.

i)Delete ii)Update iii)Alter iv) Grant v) Revoke.

- The DELETE command removes tuples from a relation.
- It includes a WHERE clause, similar to that used in an SQL query, to select the tuples to be deleted.
- Tuples are explicitly deleted from only one table at a time.
-
- The UPDATE command is used to modify attribute values of one or more selected tuples.
- As in the DELETE command, a WHERE clause in the UPDATE command selects the
- tuples to be modified from a single relation.
- However, updating a primary key value may propagate to the foreign key values of tuples in other relations if such a referential triggered action is specified in the referential integrity constraints of the DDL.

- An additional SET clause in the UPDATE command specifies the attributes to be modified and their new values.

ALTER TABLE is used to add, delete/drop or modify columns in the existing table. It is also used to add and drop various constraints on the existing table.

ALTER TABLE – ADD

ADD is used to add columns into the existing table. Sometimes we may require to add additional information, in that case we do not require to create the whole database again, **ADD** comes to our rescue.

Grant in SQL Server

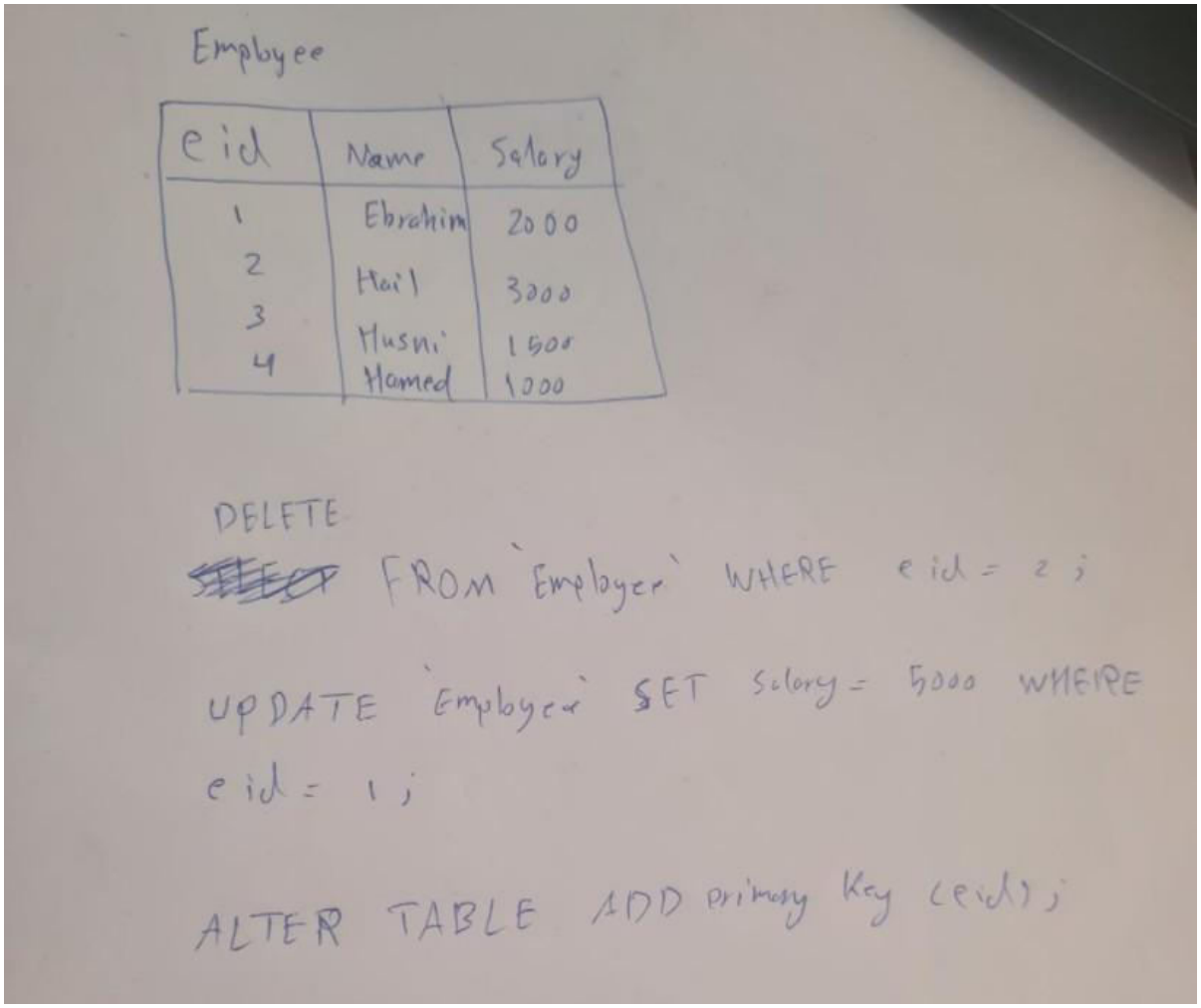
SQL Grant is used to provide permissions like Select, All, Execute to user on the database objects like Tables, Views, Databases and other objects in a SQL Server.

```
GRANT SELECT ON employee TO user1;
```

Revoke in SQL Server

SQL Revoke is used to remove the permissions or privileges of a user on database objects set by the Grant command.

```
REVOKE SELECT ON employee FROM user1
```



Q: What are database stored procedures? Give general forms for declaring a procedure.

Stored Procedures are created to perform one or more DML operations on Database. It is nothing but the group of SQL statements that accepts some input in the form of parameters and performs some task and may or may not returns a value.

Syntax : Creating a Procedure

```
  CREATE or REPLACE PROCEDURE
name(parameters) IS
  variables;
```

```
  BEGIN
  //statement
s; END;
```

The most important part is parameters. Parameters are used to pass values to the Procedure. There are 3 different types of parameters, they are as follows:

1. **IN:** This is the Default Parameter for the procedure. It always receives the values from calling program.
2. **OUT:** This parameter always sends the values to the calling program.
3. **IN OUT:** This parameter performs both the operations. It Receives value from as well as sends the values to the calling program

Q: Briefly explain Views in SQL along with the syntax. Discuss the problems that may arise when one attempts to update a view. How are views practically implemented

Views in SQL are kind of virtual tables. A view also has rows and columns as they are in a real table in the database. We can create a view by selecting fields from one or more tables present in the database. A View can either have all the rows of a table or specific rows based on certain condition.

We can create View using **CREATE VIEW** statement. A View can be created from a single table or multiple tables.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2.....
FROM table_name
WHERE condition;

view_name: Name for the View
table_name: Name of the table
condition: Condition to select rows
```

Thus problems with updating a view can be summarized as follows:

- A view with a single defining table is updatable if the view attributes contain the primary key of the base relation, as well as all attributes with the NOT NULL constraint that do not have default values specified.

- It is generally not possible to update views defined on multiple tables.
- It is not possible to update views defined using grouping and aggregate functions

Q: How are Triggers and Assertions defined in SQL? Demonstrate its practical usage with the help of example each..

What are Assertions? When a constraint involves 2 (or) more tables, the table constraint mechanism is sometimes hard and results may not come as expected. To cover such situation **S QL** supports the creation of assertions that are constraints not associated with only one table. And an assertion statement should ensure a certain condition will always exist in the database. DBMS always checks the assertion whenever modifications are done in the corresponding table.

Syntax –

```
  CREATE ASSERTION [ assertion_name ] CHECK
( [ condition ] );
```

Example –

```
    CREATE TABLE sailors (sid int,sname varchar(20), rating int,primary key(sid),
  CHECK(rating >= 1 AND rating <=10)
    CHECK((select count(s.sid) from sailors s) + (select count(b.bid)from boats b)<100) );
```

In the above example, we enforcing CHECK constraint that the number of boats and sailors should be less than 100. So here we are able to CHECK constraints of two tablets simultaneously.

2. What are T riggers? A trigger is a database object that is associated with the table, it will be activated when a defined action is executed for the table. The trigger can be executed when we run the following statements:

1. INSERT
2. UPDATE
3. DELETE

And it can be invoked before or after the event.

Syntax –

```
create trigger [trigger_name] [before | after]
{insert | update | delete} on [table_name]
[for each row] [trigger_body]
```

Example –

```
create trigger t1 before UPDATE on sailors for each row
begin
      if new.age>60 then
         set new.age=old.age; else
         set new.age=new.age; end if;
end;
$
```